Once again, terminate the previous Java process (Ctrl-C) and run the following command:

```
$ java -Dpolar.greeting="Welcome to the catalog from JVM" \
    -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar \
    --polar.greeting="Welcome to the catalog from CLI"
```

The result, as you can imagine, will be the following:

```
$ http :9001/
Welcome to the catalog from CLI
```

Both CLI arguments and JVM properties let you externalize the configuration and keep the application build immutable. However, they require different commands to run the application, which might result in errors at deployment time. A better approach is using environment variables, as recommended by the 15-Factor methodology. Before moving on to the next section, terminate the current Java process (Ctrl-C).

4.2.3 Configuring an application through environment variables

Environment variables defined in the operating system are commonly used for externalized configuration, and they are the recommended option according to the 15-Factor methodology. One of the advantages of environment variables is that every operating system supports them, making them portable across any environment. Furthermore, most programming languages provide features for accessing environment variables. For example, in Java, you can do that by calling the `System.getenv()` method.